Chapter 7

System Hardware and AI Model Training for SCARA Robot

Contents

7.1 Introduction 198

7.2 ROS Package Development for SCARA Robot 199

7.3 SCARA meta package 202

7.4 Robot Controller Hardware Design 208

7.5 Application Development and Training on HPC 213

7.6 Conclusion 217

The hardware design for the Selective Compliance Articulated Robot Arm (SCARA) robot controller involves integrating sensors, controlling actuators and managing real-time communication to support advanced motion control. This chapter focuses on developing a hardware architecture that interfaces efficiently with software frameworks such as ROS1, ros_control and MoveIt. The hardware design is guided by prior knowledge to optimize system performance and minimize errors. The chapter details the ARM-based controller, emphasizing the interaction between the hardware interface and control loops. It also discusses how minor changes in sensor or actuator configurations may require system adjustments. Additionally, the chapter explores the use of a HPC system for training generative AI models to enhance robot trajectory prediction. These models leverage spatial, spatio-temporal and attention-based learning techniques to improve motion planning and control. The aim is to ensure a robust and adaptive system by combining real-time hardware control with AI-driven trajectory generation.

7.1 Introduction

Hardware implementation focuses on the architecture of the robot controller designed for a SCARA robot. This includes integration of control hardware, communication interfaces and system-level interconnections necessary to satisfy real-time performance requirements. At the core of the implementation is the Embedded Robot Controller, which interfaces directly with the SCARA robot. It handles time-critical tasks such as low-latency motor actuation, sensor data acquisition and real-time control loop execution under deterministic timing constraints. A host workstation is connected to the Embedded Robot Controller through a UART serial communication link. This workstation executes a ROS node that governs high-level control logic. The node issues motion commands, processes feedback and coordinates state transitions essential for robot task execution and supervisory management.

An additional computational resource, the HPC Facility, provides infrastructure for offloading intensive workloads. The HPC environment typically comprises GPU-accelerated compute nodes, high-speed network-accessible storage and a centralized management station. These elements are interconnected through a secure, isolated network segment, governed by access filters and routing elements positioned between public-facing nodes and internal computing resources. Fig. 7.1 illustrates the network connectivity setup used in the department. It includes the robots, their control units, a high-performance computing facility and other related computational infrastructure.

A dedicated high-speed network cable, routed through an isolated and secure router, interconnects the HPC Facility, the Robotics Lab and other major research laboratories that rely on high-performance computing. This private network segment is physically and logically separated from general-purpose institutional traffic to prevent congestion and ensure low-latency, high-throughput data exchange. By isolating critical research communication from public or administrative networks, the setup not only enhances security but also guarantees consistent performance for real-time robotic control and large-scale computational tasks.

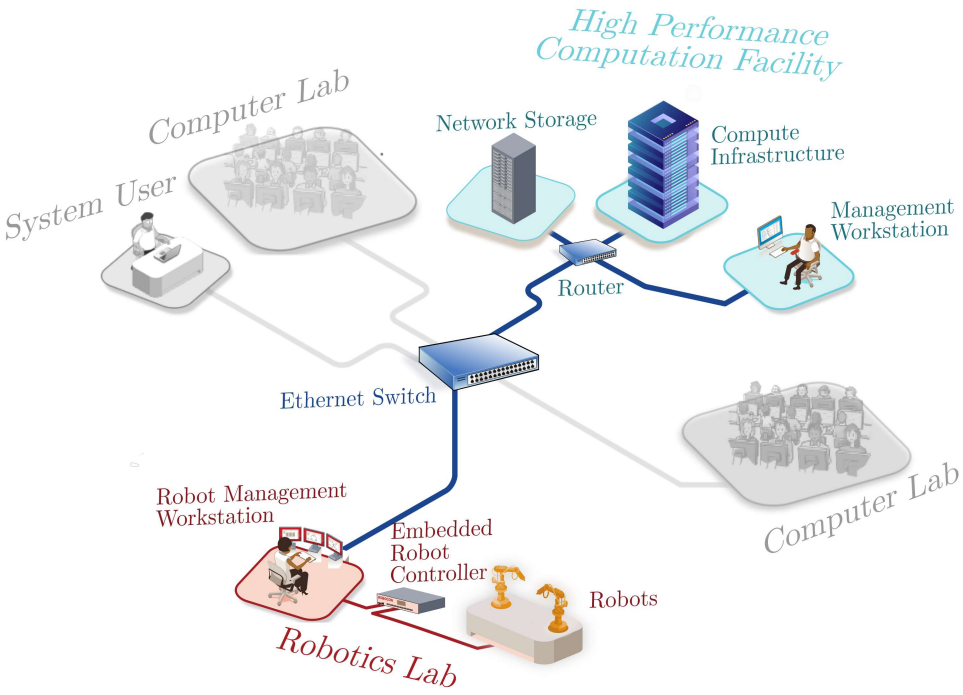


Fig. 7.1: The network connectivity between various hardware components is illustrated in this figure, including the Robotics Lab, which comprises the Robot Management Workstation, Embedded Robot Controller and Robots. The HPC Facility features a router for isolating web traffic, network storage for high-speed data loading, GPU-powered compute infrastructure and a management workstation for executing and managing HPC computations. Additionally, connections to Computer Labs and System Users are depicted for contextual understanding.

7.2 ROS Package Development for SCARA Robot

SCARA robots are extensively used in industrial automation due to their high-speed, precision and reliability. These robots excel in tasks requiring horizontal motion, such as pick-and-place operations, assembly, packaging and material handling. The structural configuration consists of two parallel rotary joints, allowing movement in the horizontal plane while maintaining rigidity along the vertical axis. This design ensures minimal positioning errors and high accuracy, making SCARA robots ideal for applications where repeatability is critical. Over time, these robots have become integral to industries such as electronics, automotive and pharmaceuticals due to their efficiency and ease of integration into automated production lines.

Developing a 3D simulation model is essential for designing and implementing robotic systems. A well-structured model facilitates visualization of the mechanical

structure, analysis of kinematics and dynamics and performance evaluation before deployment. Simulation platforms such as Gazebo and RViz, integrated with ROS, provide realistic simulations by incorporating physics engines, collision detection and sensor feedback. These tools ensure accurate representation of the robot's behaviour in a controlled environment, enabling early identification and correction of potential errors. Additionally, AI and ML techniques enhance robotic control systems by enabling adaptive behaviour, optimizing motion planning and improving fault detection through data-driven learning processes.

7.2.1 Selective Compliance Assembly Robot Arm

The Rhino SCARA robot, characterized by its versatile design, encompasses four axes tailored for education, training and research applications. Comprising three revolute joints and one prismatic joint, all axes operate independently, affording a high degree of control and flexibility. The robot has a gripper, adding an extra dimension to its functionalities. Standing at a length of 24 inches and a height of 18.75 inches, this compact yet robust system is equipped with five permanent magnet direct current (PMDC) servo motors featuring integral gearboxes and incremental optical encoders, ensuring real-time closed-loop operation. The work envelope spans 18 inches in reach, with an outer radius of 18 inches and an inner radius of 9 inches. Noteworthy are the impressive rotation capabilities, with Motor "E" providing 220 degrees of shoulder rotation, Motor "D" offering 220 degrees for elbow rotation, Motor "B" contributing 275 degrees for wrist rotation and Motor "C" facilitating a 3.75 inches total vertical travel. This Rhino SCARA model stands as a sophisticated platform for diverse applications, demonstrating precision and adaptability in its articulated movements.

Table 7.1: Denavit–Hartenberg parameters of the Rhino SCARA robot. The table presents the link index i, link length a_i, link twist angle α_i, link offset d_i and joint angle variable θ_i for each joint of the SCARA robot.

Link i	a_i	α_i	d_i	θ_i
Link 1	a_1	0	0	q_1
Link 2	a_2	0	0	q_2
Link 3	0	π	d_3	0
Link 4	0	0	0	q_4

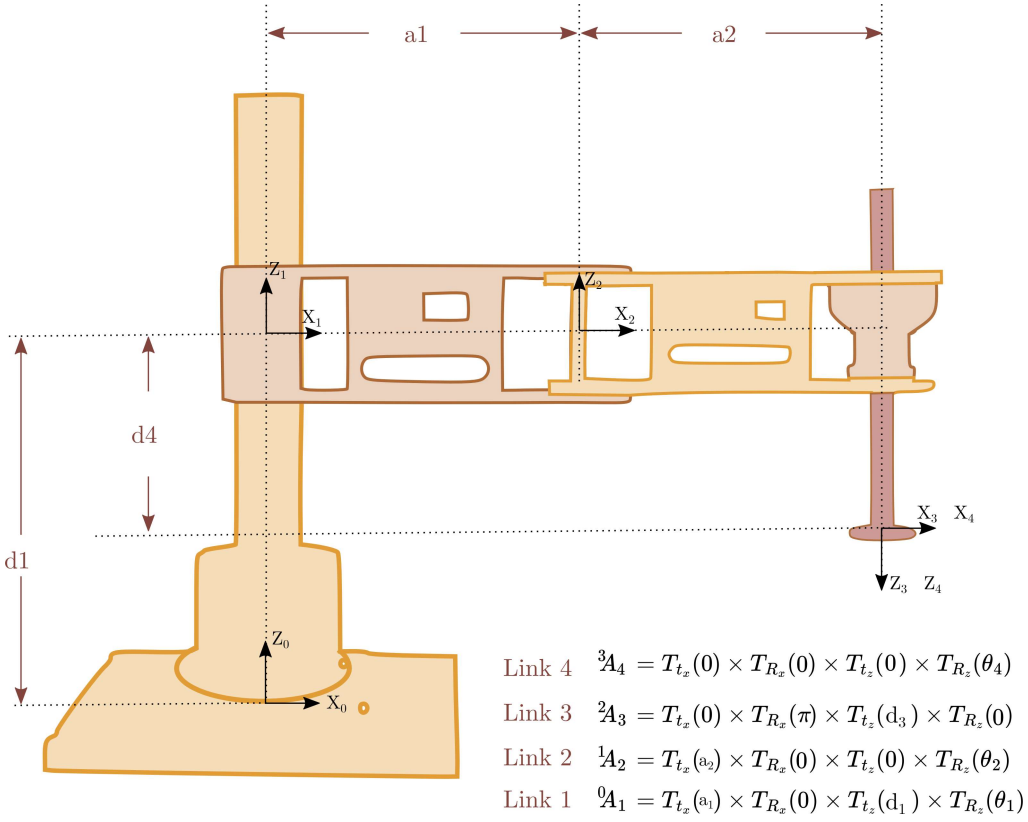


Fig. 7.2: Kinematic model of the SCARA robot arm. The figure illustrates the 4-degree-of-freedom manipulator with labelled joint frames and link equations. Each joint is represented with its corresponding coordinate frame and link length. Key measurements and parameters are indicated, providing a comprehensive view of the robot's kinematic structure and joint configurations.

The SCARA robot used in this study consists of two rotational joints and one prismatic joint that enables vertical movement. The arm structure is designed to perform tasks that require high precision and speed in a 2D plane, such as pick-and-place, assembly and packaging. The kinematic model of SCARA arm is visualised in Fig. 7.2 and the corresponding DH parameters are explained in Table 7.1.

7.2.2 Introduction to ROS Framework and Architecture

The Robot Operating System is a modular, open-source framework that provides a set of tools and libraries for developing, simulating and deploying robotic systems. ROS facilitates communication between different components of a robotic system by using a publisher-subscriber architecture, where nodes exchange messages through defined topics. Each ROS node performs a specific task, such as reading sensor data, controlling motors, or executing high-level decision algorithms.

In the context of the SCARA robot, ROS is used to manage various subsystems, including motion control, inverse kinematics and sensor integration. ROS also enables seamless interaction between simulation environments (such as Gazebo) and hardware interfaces, allowing for rapid prototyping and real-time testing.

7.3 SCARA meta package

The design and organisation of the ROS package structure for the SCARA robot involve the creation of a meta package that encompasses multiple individual packages, each serving a specific purpose. The objective is to provide a modular and well-structured framework for the development, simulation, control and hardware interface of the SCARA robot using ROS, Python and CMake.

- **scara_description** - This package is dedicated to the description of the SCARA robot's URDF [358], 3D meshes and xacro files. It plays a crucial role in defining the robot's kinematic and dynamic properties in a virtual environment.
- **scara_moveit** - The scara_moveit package integrates motion planning algorithms seamlessly with the SCARA robot. It incorporates collision detection and visualization capabilities through MoveIt, providing a comprehensive environment for planning and executing motion trajectories.
- **scara_gazebo** - Gazebo simulation is facilitated by the scara_gazebo package, which acts as a bridge between ROS and Gazebo. This package enables the integration of the SCARA model into the Gazebo simulator, allowing developers to validate and test algorithms in a realistic virtual environment.
- **scara_control** - The scara_control package is designed for communication purposes with the SCARA robot. It implements control algorithms and functionalities, ensuring effective and reliable interaction between the software components and the physical robot in the laboratory setting.
- **scara_hw** - Hardware interface is a critical aspect of robot control and the scara_hw package addresses this by providing a comprehensive hardware interface for the SCARA robot. It incorporates plugins that facilitate communication

between the ROS framework and the robot's physical components, ensuring a seamless connection between software and hardware.

7.3.1 SCARA Description Package

The *scara_description* package serves as the core module responsible for encapsulating the structural and kinematic configuration of the SCARA robotic manipulator. At the heart of this package lies the URDF file, which is pivotal in defining the robot's mechanical structure in a platform-agnostic manner. This XML-based declarative specification provides a detailed representation of the robot's physical architecture, including link dimensions, joint types, mass properties, inertial parameters and coordinate frames. URDF generation often begins with exporting detailed 3D models from Computer Aided Design (CAD) software, streamlining the transition from mechanical design to robotic simulation and control integration.

Blender [359] is a free and open-source 3D modelling software that excels in creating 3D objects from vector files. With its extensive suite of tools and features, Blender allows users to import 3D vector files in various formats and transform them into intricate and detailed 3D objects. Its user-friendly interface, along with advanced functionalities like mesh manipulation and texturing, empowers designers and artists to bring their vector-based concepts to life in the three-dimensional world. This flexibility and versatility make Blender an invaluable tool for those looking to bridge the gap between vector design and 3D object creation. The Graphical User Interface of Blender with SCARA robot is visualised in Fig. 7.3.

Robot simulation serves as an invaluable tool in the field of robotics for cost reduction, Risk Mitigation, Rapid Prototyping, Accessibility, Algorithm Development, Testing and Validation, Scale and Reproducibility etc. The development of Robot simulation need the 3D models of Computer Aided Designs. Blender is utilised as a platform for developing the CAD designs. Each link is designed separately by measuring the real robot. Fig. 7.4 visualizes Link 1 of the SCARA robot with multiple standard CAD views, including front, top, side, and isometric perspectives. The model is visualised along with the mounted motors. Similarly the Link2 is portrayed in Fig. 7.5.

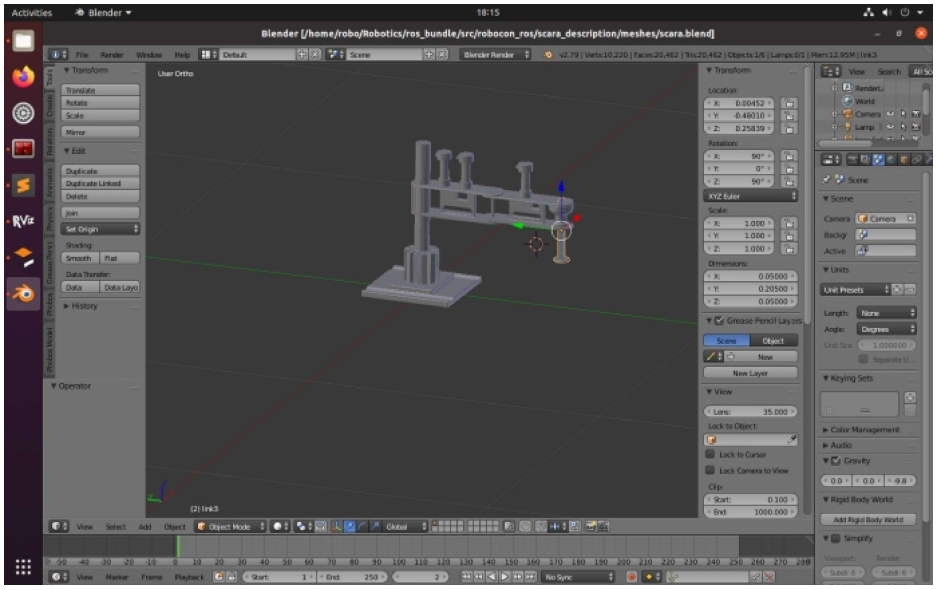


Fig. 7.3: The Graphical User interface of SCARA robot using Blender

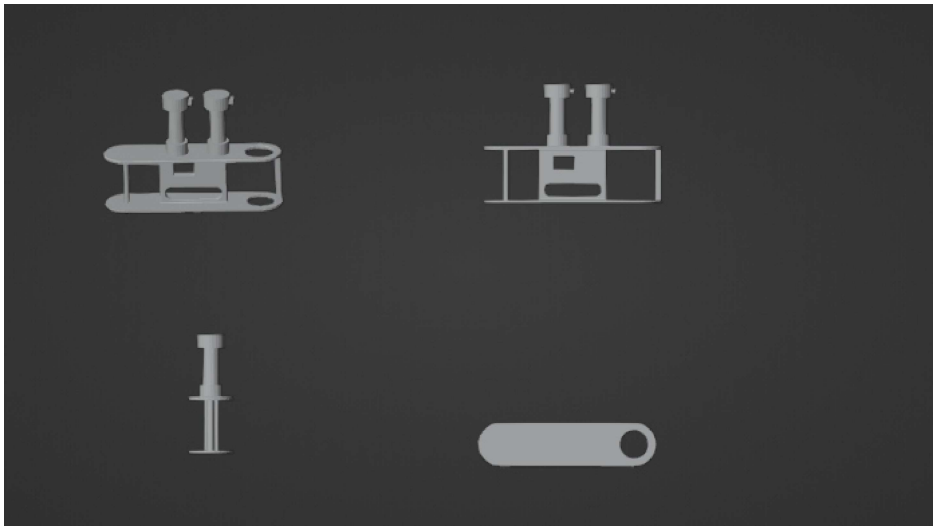


Fig. 7.4: SCARA Link1 CAD file with top, side, front and rotated views

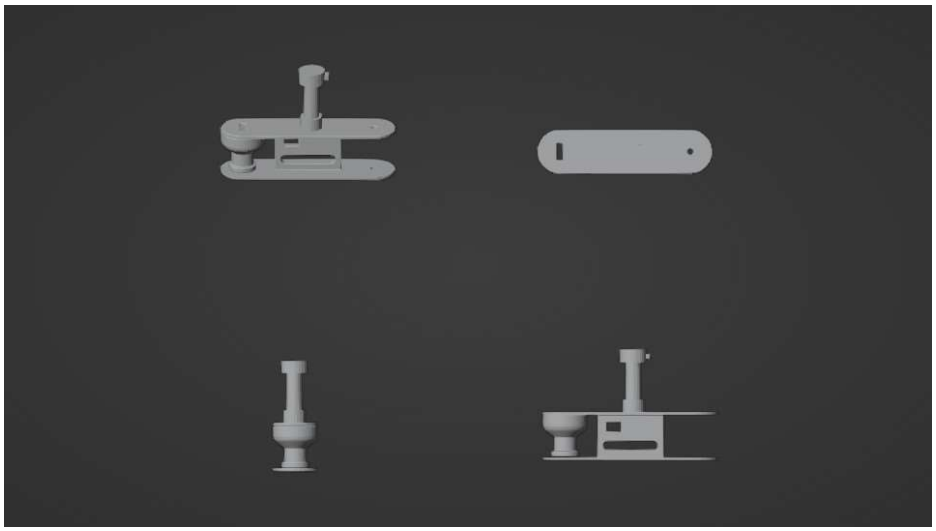


Fig. 7.5: SCARA Link2 CAD file with top, side, front and rotated views

The rendered version of robot in Blender is visualised in Fig. 7.6. The model is rendered with an artificial light source and a virtual camera interface.

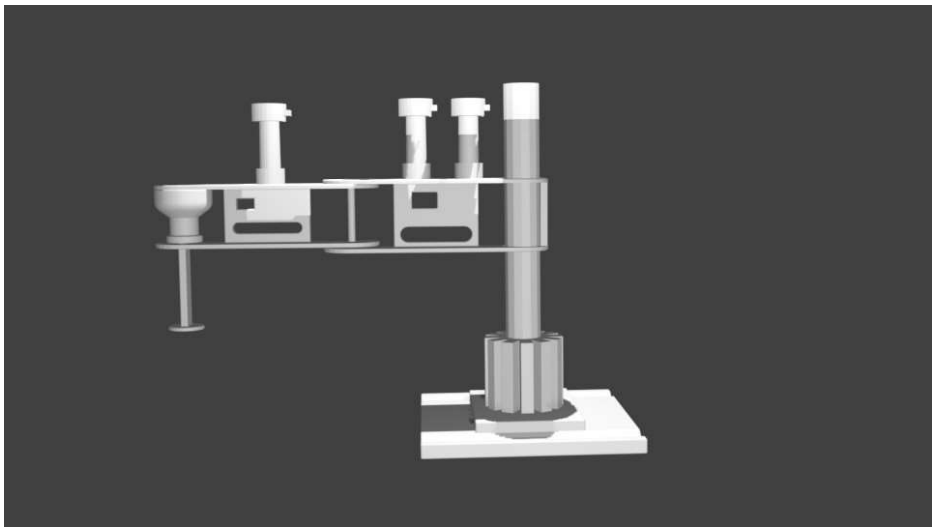


Fig. 7.6: Rendered SCARA Robot visualisation in Blender

To integrate the 3D model with ROS, the CAD design was converted into aURDF file. URDF, an XML-based specification native to ROS, is used to define a robot's kinematic structure and dynamic properties. The conversion process entailed exporting the CAD model into a compatible format such as STL or Collada, followed by the generation of a URDF file that explicitly defines the robot's links, joints and inertial parameters. Additionally, visual and collision geometries were associated with the model to ensure accurate rendering and physical interaction in simulation. This

finalized URDF file was then employed to visualize and validate the SCARA robot's configuration and behaviour within simulation platforms.

7.3.2 SCARA Moveit Package

The `scara_moveit` package serves as the motion planning and manipulation interface for the SCARA robot within the ROS ecosystem, leveraging the capabilities of the MoveIt framework. This package encapsulates critical components such as the robot's planning group configuration, kinematic solvers, collision geometry and trajectory execution interfaces. Through integration with the URDF and Semantic Robot Description Format (SRDF), `scara_moveit` enables interactive motion planning, inverse kinematics and real-time visualization in environments like RViz. It facilitates the generation and execution of collision-free trajectories, allowing the SCARA robot to perform precise pick-and-place operations, path interpolation and joint-space or Cartesian-space motions. Additionally, the package supports configuration for controllers and planning pipelines, enabling seamless simulation-to-hardware transitions. Overall, the `scara_moveit` package forms a vital bridge between high-level motion planning algorithms and low-level actuator commands, ensuring accurate, safe and efficient robot manipulation.

7.3.3 SCARA Gazebo Package

The `scara_gazebo` package provides a simulation environment for the SCARA robot using the Gazebo physics engine, enabling dynamic testing and validation of control algorithms in a virtual setting. This package integrates the robot's URDF model with Gazebo-specific plug-ins and configuration files, including transmission interfaces, joint controllers and sensor emulators. It allows for accurate physical simulation of joint dynamics, actuator responses and environmental interactions under real-world conditions. The package also includes launch files to spawn the robot model into the Gazebo world, load the appropriate controllers and initiate joint state broadcasting. Through the `scara_gazebo` package, developers can perform comprehensive testing of trajectory execution, collision detection and sensor feedback mechanisms without risking hardware damage. Moreover, it supports synchronized operation with MoveIt for end-to-end simulation of perception, planning and control pipelines. This

modular simulation environment is crucial for iterative development, performance tuning and system-level validation of the SCARA robotic platform.

7.3.4 SCARA Control Package

The `scara_control` package is responsible for managing the low-level actuation and joint-level control of the SCARA robot within the ROS control framework. It incorporates configuration files and controller plug-ins that interface with hardware or simulation back-ends via the ROS `controller_manager`. The package defines position, velocity and effort controllers for each joint, enabling precise motion execution in response to trajectory commands. It utilizes the `hardware_interface` abstraction layer to communicate with the custom embedded system, facilitating real-time actuation through the ARM Cortex-M3-based controller. Additionally, this package supports both simulated and physical hardware configurations by allowing seamless switching between `gazebo_ros_control` plugins and physical hardware interfaces. The `scara_control` package is integral to ensuring stable, responsive and accurate joint behaviour, providing the execution backbone for higher-level motion planning frameworks like MoveIt. It enables real-time feedback, control loop execution and smooth trajectory tracking essential for the robot's functional deployment.

7.3.5 SCARA Hardware Package

The `scara_hw` package implements the hardware abstraction layer that bridges the ROS control architecture with the SCARA robot's embedded controller. It defines a custom hardware interface by utilizing a UART communication link between the host PC and the embedded controller, operating at a baud rate of 19200 bits per second (bps). This interface enables bidirectional communication where high-level command signals, effort for each joint, are transmitted from the ROS controller and the embedded system responds with the current absolute joint positions.

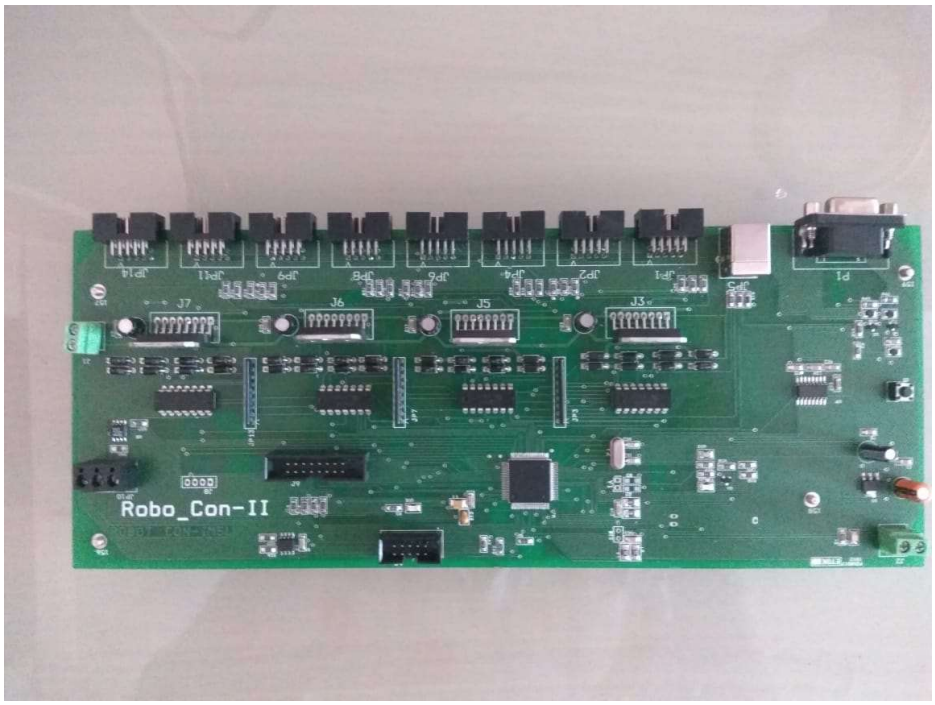
The embedded controller interprets quadrature encoder signals—specifically the A and B phase pulses—from each joint's motor to maintain an accurate count of the joint's absolute position. These encoder counts, which represent discrete position states, are continuously tracked and transmitted back to the PC. Within the `scara_hw` node, these raw count values are converted into joint angles by applying a scaling factor based on the encoder resolution and the mechanical configuration of the joints. This

conversion facilitates the mapping of hardware-specific positional data into meaningful angular values in radians, as required by the ROS control framework. The package ensures that real-time feedback and command execution are synchronized, enabling precise, deterministic and robust control of the SCARA robot within both simulation and physical environments.

7.4 Robot Controller Hardware Design

The robot controller hardware is built upon a custom-designed Printed Circuit Board (PCB) featuring the ARM Cortex-M3 (LPC1768) microcontroller which is illustrated in Fig. 7.7. The LPC1768 offers a balanced combination of processing power, peripheral integration and real-time performance, making it highly suitable for embedded control applications. Specifically, it operates at clock frequencies up to 100 MHz, which enables deterministic execution of control loops, sensor fusion algorithms and trajectory generation tasks. Its Cortex-M3 core is optimized for low-latency interrupt handling, crucial for responsive actuator control and time-critical event management in robotics. The sophisticated setup offers robust computational capabilities within a compact form factor. The controller supports both Universal Serial Bus (USB) and RS232 communication protocols, enabling versatile connectivity options with the host PC for data exchange and control. The LPC1768 chip provides ample storage with 512KB of flash memory for program storage and 64KB of data memory, ensuring efficient execution of complex control algorithms. For software development and device programming, the Keil µVision4 integrated development environment is employed, offering a comprehensive suite of tools for code writing, debugging and optimisation. The firmware is flashed onto the device using Flashmagic, a user-friendly utility that facilitates quick and reliable program updates. This carefully crafted hardware-software combination results in a flexible and powerful robot control system capable of meeting the demands of advanced robotic applications.

The robot is actuated with DC permanent magnet motors. Four L298 motor driver chip and associated circuitry are used for control. These drivers require two input signals and an enable. Speed control of joint movements is achieved using Pulse Width Modulation (PWM) signals. Additional ports are provided for auxiliary



(a)



(b)

Fig. 7.7: The figure showcases a custom-developed robot controller system. (a) Depicts the complete controller unit, while (b) illustrates its corresponding PCB layout. Powered by an ARM LPC1768 microcontroller, this controller features 8 ports for interfacing with various robotic components. These ports enable control of motors in robot joints and associated structures within the work cell, such as conveyor belts, enhancing overall automation capabilities.

systems like conveyor belt, rotating table etc. The motor module contains a permanent magnet DC motor, limit switches, home switch, feedback and power circuitry. The connection to the module contains supply lines, home switch, actuator controls and feedback lines. Two optical proxy sensors are placed 90-degree phase shifted to get quadrature encoded A & B pulses at the output as shown in Fig. 7.8. Limit switches are connected for enhanced safety. Each motor and its controller provides A and B

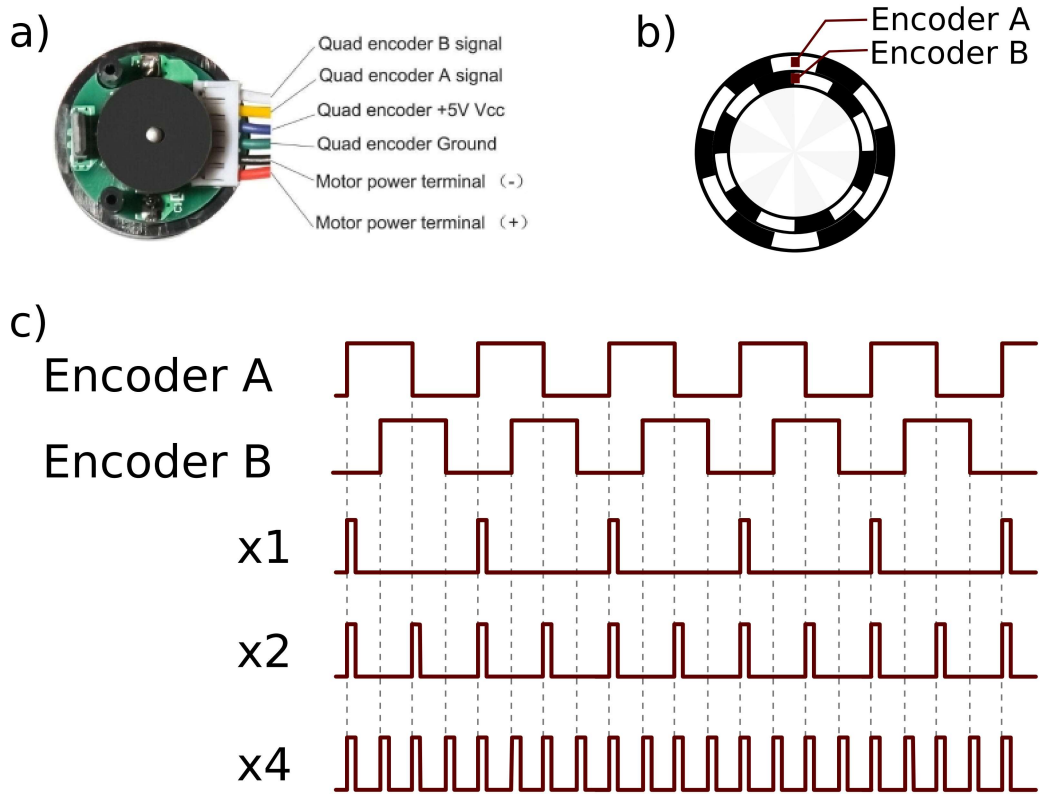


Fig. 7.8: Quadrature encoder system utilized in the robotic motor feedback mechanism. (a) Labelled view of the hardware PCB showing the quadrature encoder interface, including input terminals for channels A and B, as well as power and ground connections. (b) Optical encoder disc employed for generating channel A and B signals, enabling the measurement of angular position and direction of motor shaft rotation. (c) Timing diagram depicting quadrature signals A and B, illustrating the relative phase shift and rising edges used for decoding schemes including X1 (single-edge), X2 (dual-edge) and X4 (quad-edge) decoding, which enhance resolution in position tracking and motion control algorithms.

pulses from quadrature encoder. The different channels in the controller is configured independently. The controller board can be utilised for any robot which can actuate upto 8 motors.

The controller software, developed in Keil µVision4, is designed with backward compatibility in mind to support a range of hardware configurations. A critical component is the pulse counter, implemented as an interrupt-driven system for precise motor position tracking. This counter translates encoder pulses into angular measurements, providing accurate degree-based position feedback. Home switches play a vital role in establishing a reference point for the robot's axes, ensuring consistent and reliable positioning across power cycles. The software also includes stall detection, hard-home and control mechanisms, which monitor motor current and position to identify and respond to unexpected resistance or immobility. This comprehensive approach to software design results in a controller capable of precise motion control, self-calibration and fault tolerance, essential for reliable robotic operations.

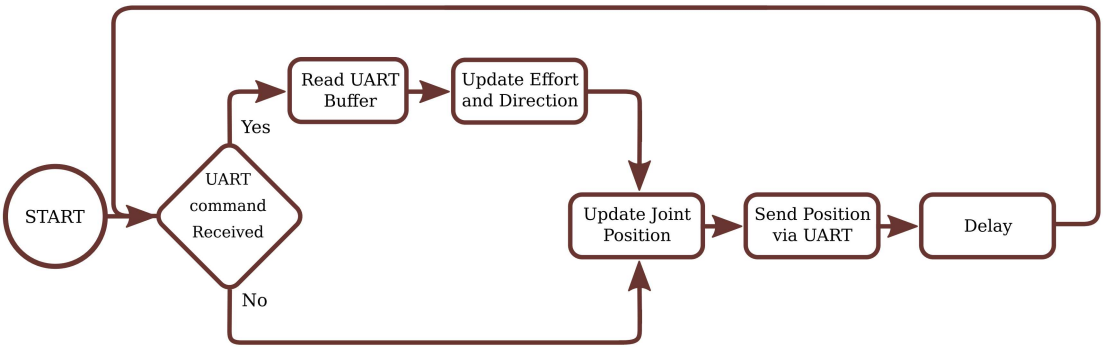


Fig. 7.9: Block Diagram of Control Loop in LPC1768 for SCARA Robot.

The "hard-home" refers to the process of moving a robot or machine axis to its absolute home position using physical limit switches or dedicated homing sensors (rather than relying solely on software or encoder counts). Hard-homing is implemented in SCARA robot, where joints move in one direction until reaching a mechanical limit, then stop and reverse at a slower speed to precisely trigger a limit switch. As the axis passes the switch during this controlled backward movement, the system stops and resets the robot position as the absolute zero. This two-stage approach ensures accurate homing while preventing mechanical stress. Once established, encoders track all subsequent movements relative to this home position. The method provides reliable, repeatable positioning by physically verifying the reference point. If power is lost, the system must re-home since standard encoders don't retain position

information without this physical reference. The overall sequence of the hard-homing process implemented in the SCARA robot is illustrated in Fig. 7.10.

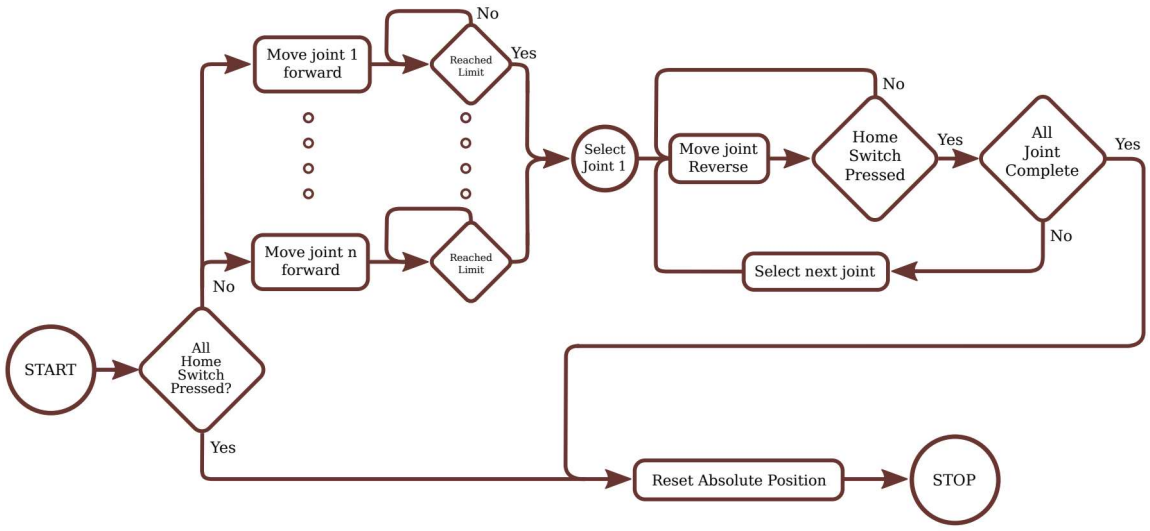


Fig. 7.10: Block Diagram of Hard-Homing Sequence for SCARA Robot.

The robot is equipped with PMDC motors that utilize quadrature encoders. The system monitors motor movement and applied effort to determine whether each motor is rotating as expected when power is applied. The quadrature encoder's A and B pulses provide real-time position feedback, allowing velocity estimation by tracking position changes over time. A stall condition is detected when the motor's effort exceeds a defined threshold while the position remains nearly unchanged for a specified duration. An ARM timer module is used to measure the elapsed time during high-effort, low-movement conditions, triggering a stall response if the timeout period is reached. Additionally, limit switches and a home switch help distinguish between expected stops (such as reaching a mechanical limit) and unexpected stalls. The algorithm ensures robust detection while minimizing false positives, enabling the controller to halt the motor or initiate recovery procedures when a stall is confirmed. This method enhances system reliability by preventing motor overheating and mechanical stress during blocked or overloaded conditions.

7.4.1 System Limitations

The system having been purchased in the early 1990s, introduces inherent limitations due to technological obsolescence. A primary concern is motor slipping, which can occur due to wear in mechanical components, leading to inaccuracies in positioning and reduced overall precision. The effects of aging are evident across various system components, potentially compromising reliability and efficiency. Furthermore, the control architecture is limited to position control only, lacking more advanced control strategies such as velocity or torque control. This restriction in control capabilities constrains the system's adaptability to diverse tasks and environments. The combination of these factors—motor slipping, aging hardware and limited control options—significantly impacts the system's accuracy, versatility and ability to meet modern robotic performance standards.

7.5 Application Development and Training on HPC

High Performance Computing refers to the practice of aggregating computing power to deliver far higher performance than is possible with typical desktop computers or workstations. HPC systems utilises parallel processing techniques to solve complex computational problems across a wide range of disciplines, including scientific research, engineering, finance and big data analytics.

7.5.1 HPC architecture

Fig. 7.11 illustrates the comprehensive HPC network architecture, detailing its internet connectivity, security features, management components, high-performance accelerator module, storage capabilities, low-latency interconnects and administrative infrastructure, all of which collectively contribute to the system's robust computational capacity and efficient operation.

The software architecture of HPC is illustrated in Fig. 7.13. The *Container I* in the figure is utilised for disentanglement model training in this work by utilising tensorflow framework. *Container II* is utilised for all other implementations by utilising Pytorch Lightning framework with Pytorch Kinematics package for computing the robot kinematics computation on CUDA cores.

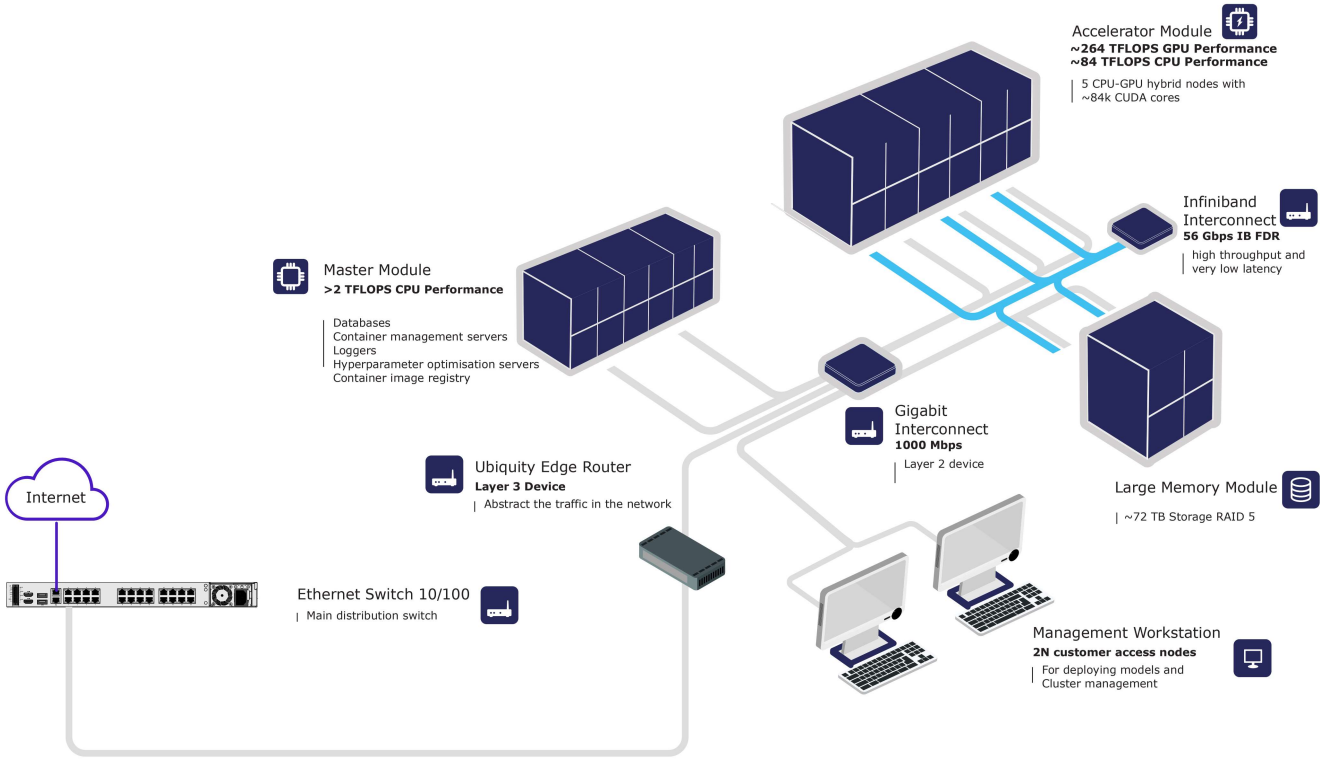


Fig. 7.11: HPC Network Architecture featuring Internet connectivity through the gateway. A Layer 3 router mitigates packet collisions within the network and enhances security measures. The Master module encompasses essential tools, loggers and databases for efficient management and utilization. The Accelerator module is empowered by CUDA-enabled systems, delivering an aggregate performance exceeding 300 TFLOPS. Storage nodes equipped with large memory modules ensure high throughput. Infiniband interconnect facilitates low-latency data transfer between nodes. Additionally, a management workstation is provided for node management and deployment tasks.



Fig. 7.12: High-Performance Computing Stack: Featuring compute nodes, master nodes, storage nodes and network stack seamlessly integrated for optimal performance and scalability.

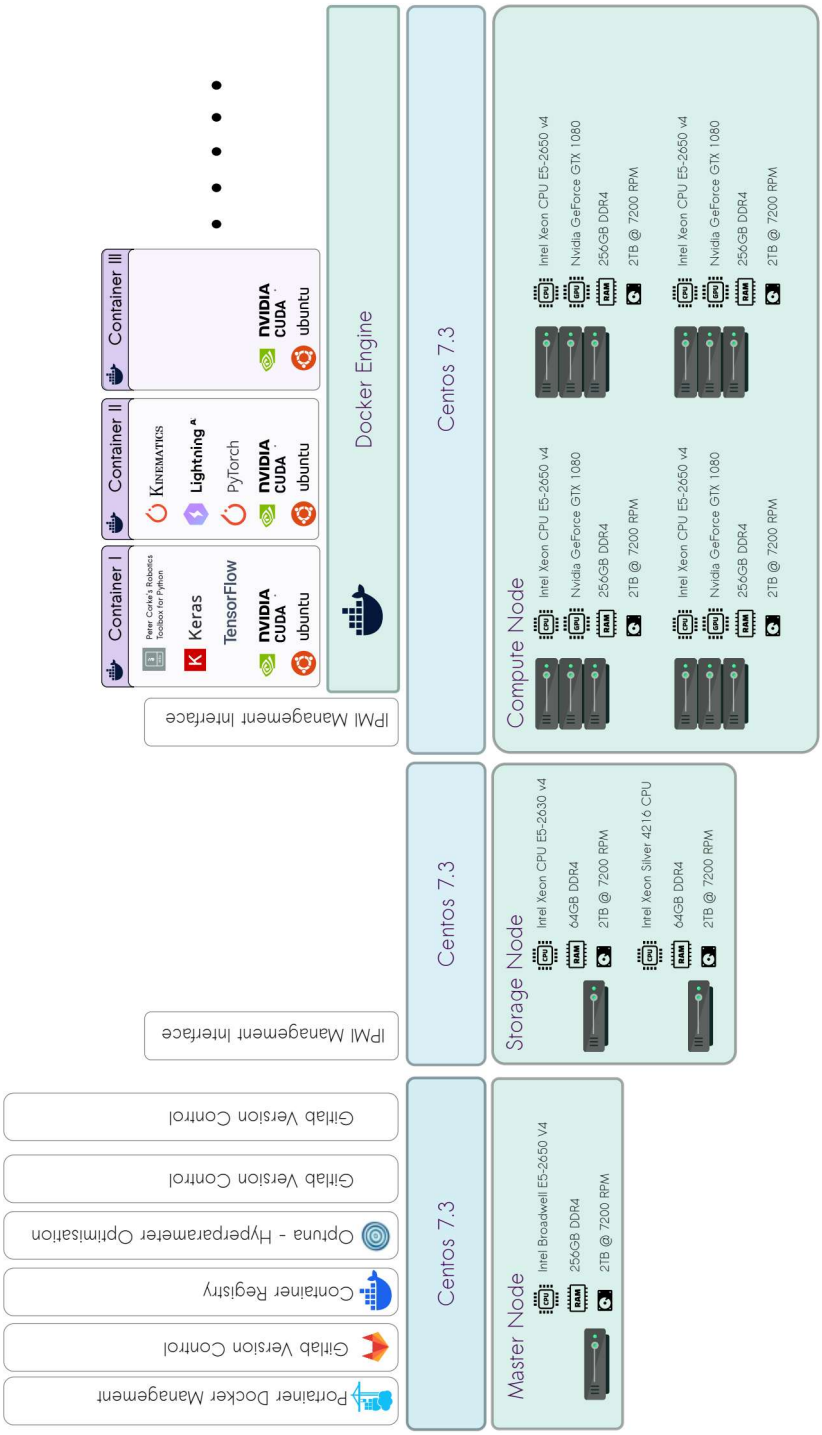


Fig. 7.13: Software architecture of the High Performance Computing (HPC) facility, highlighting its hardware specifications. The architecture comprises the following components - **Master Node**: This node serves as the central hub for managing Docker containers, GitLab version control, hyperparameter optimisation algorithms and other essential functionalities. **Storage Node**: Dedicated for storing large, high-fidelity datasets, ensuring efficient access for computations. **Compute Nodes**: Distributed across various systems, these nodes uses Docker containers orchestrated by a container orchestration engine to execute intensive workloads.

Table 7.2 clearly conveys that the key specification of the HPC architecture being discussed is its overall computational power, which is stated as 352 TFLOPS and Fig. 7.12 shows the HPC stack utilised for the work. TFLOPS is a common metric used to measure and compare the performance of high-performance computing systems, especially for scientific and engineering applications that involve intensive numerical calculations.

Table 7.2: HPC System Specifications: GPU, CPU and Computational Performance

Specification	Count	TFLOPS
Intel Xeon CPU E5-2650 (CPU)	14	84 TFLOPS
Nvidia GeForce GTX 1080 Ti (GPU)	20	226 TFLOPS
Nvidia Pascal 100 (GPU)	4	42.4 TFLOPS

7.6 Conclusion

This chapter presented the embedded controller development process of the SCARA robot, including 3D simulation model creation, ROS package development, AI model training and evaluation on real hardware. The simulation environment, built using ROS and Gazebo, provided an accurate representation of the robot's kinematic and dynamic behaviour, enabling effective validation of control algorithms and AI models before deployment.

The ROS packages developed for the SCARA robot facilitated seamless communication between hardware components and software modules, ensuring smooth task execution. AI models, trained using data collected from the simulation, demonstrated high prediction accuracy, adaptability to environmental changes and efficient trajectory correction. The deployment and evaluation of these models on real hardware highlighted their effectiveness in improving task performance, with minimal deviation between simulated and real-world results.